

Fine-Tuning LLMs for Bug Detection in Quantum Software

CSC 7644 - Applied LLM Development | Final Project Report | Md Saidur Rahman

GitHub Link: <https://github.com/saidur-rahman/csc7644-final-project-rahman>

Abstract

Quantum software development introduces domain-specific defects such as incorrect qubit mappings, gate misuse, measurement-ordering errors, and circuit-construction mistakes, whose triage requires both software engineering and quantum computing expertise. This work fine-tunes and evaluates transformer-based language models for bug detection in quantum software, focusing on binary classification of Qiskit-centered bug reports as *classical* or *quantum-specific*. Three RoBERTa-architecture encoder backbones, roberta-base, codebert-base, and graphcodebert-base, were trained under an identical protocol so that the effect of the pre-training corpus could be compared fairly. Each backbone was evaluated using stratified 5-fold. Cross-validation was repeated across five random seeds, producing 25-fold runs per backbone and 75-fold runs in total. Across the three models, mean accuracy ranged from 0.758 to 0.767, and mean ROC-AUC was approximately 0.86. CodeBERT achieved the highest mean accuracy and macro-F1, while GraphCodeBERT achieved the highest mean ROC-AUC. The implementation uses HuggingFace Transformers, PyTorch, and scikit-learn, and the code, evaluation outputs, and figures are released publicly through the project. GitHub repository. Overall, this work demonstrates a reproducible LLM fine-tuning pipeline for quantum software bug detection and highlights the importance of careful task framing, class-imbalance handling, and repeated evaluation in applied LLM development.

1. Introduction and Background

Quantum computing is rapidly transitioning from theoretical research to practical software development. Frameworks such as IBM's Qiskit allow developers to write and execute quantum programs on real hardware and simulators. Quantum programs, however, are uniquely susceptible to defects with no analogue in classical software: incorrect qubit mappings, gate misuse, measurement ordering errors, and entanglement violations. These quantum-specific faults are difficult to surface with traditional debugging tools, which lack awareness of quantum semantics [7]. Today, triage of such bugs is performed manually by quantum software engineers a labour-intensive process that requires deep domain expertise and does not scale with the growing volume of quantum software.

Pre-trained language models such as CodeBERT [3] and GraphCodeBERT [4] have demonstrated strong performance on classical software-engineering tasks including bug detection, code summarisation, and program repair. Fine-tuning a pre-trained encoder on labelled quantum bug data allows the model to internalise domain patterns directly into its weights, rather than relying on inference-time prompting or retrieval. A reliable classifier of this kind could serve as a triage layer that routes incoming bug reports to the appropriate engineering team, shortening the path from defect report to diagnosis and reducing the cognitive load on quantum develop-

ers. This project, the final deliverable for CSC 7644, fine-tunes and compares three pre-trained encoders that share architecture and parameter count but differ in pre-training corpus, isolating the effect of corpus choice on classical-versus-quantum discrimination.

2. System Design and Implementation

2.1 Architecture and Design Decisions

The system adopts a sequence-classification architecture: a frozen or fine-tuned transformer encoder whose [CLS] representation is fed into a two-class linear head. Three encoder backbones were chosen to isolate the effect of pre-training corpus while holding all other variables fixed: `roberta-base` [6] (natural language only, control), `codebert-base` [3] (code + natural language), and `graphcodebert-base` [4] (code + natural language + data-flow graphs). All three share the RoBERTa architecture and approximately 125M parameters, so any performance difference can be attributed solely to the pre-training corpus. The end-to-end methodology pipeline is shown in Figure 1.

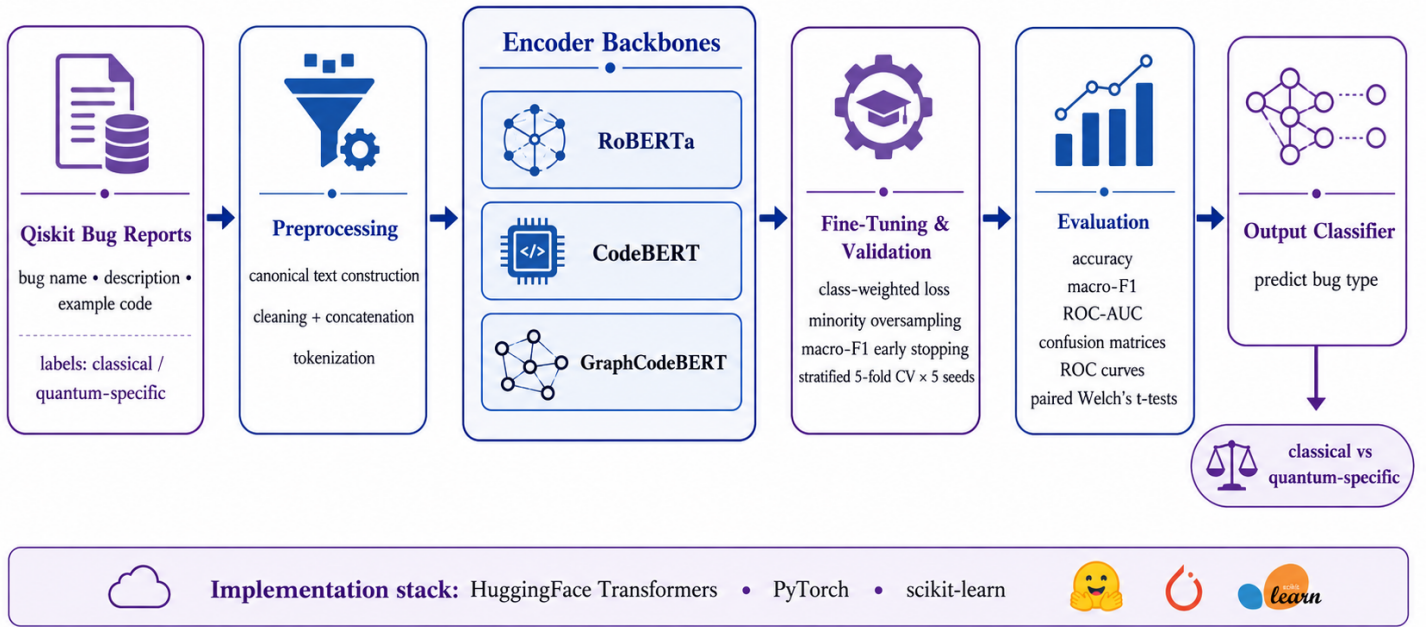


Figure 1: End-to-end methodology pipeline

2.2 Data Sources and Preprocessing

The labelled corpus was drawn from the Bugs-QCP collection (Zenodo 5834281) [1], filtered to retain only samples explicitly labelled as classical or quantum. The final set contained $n = 233$ samples (134 classical, 99 quantum), shown in Figure 2. Each bug entry was converted into a single canonical string by concatenating the [Bug Name], [Description], and [Example Code] fields with newline separators; empty fields were dropped. The string was then tokenised with the backbone’s pretrained tokeniser and truncated to 256 tokens, preserving the full prose for the vast majority of samples. A signed-stamped split manifest was committed to

version control to prevent leakage between folds.

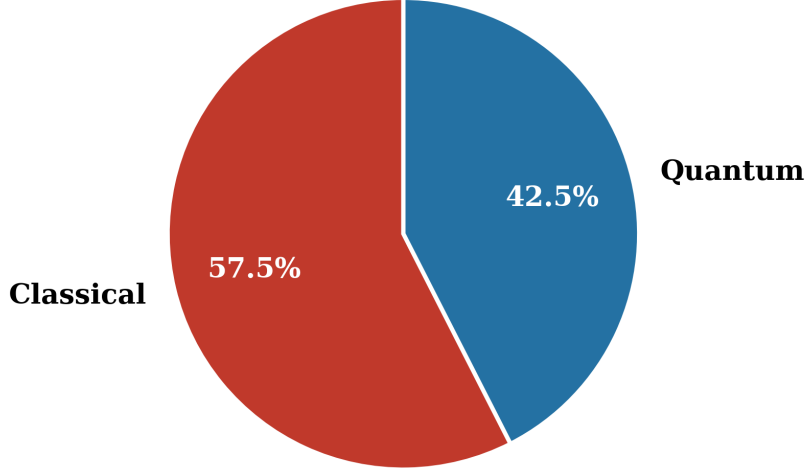


Figure 2: Class distribution of labeled Qiskit bug samples

2.3 Model Selection

To isolate the contribution of pre-training data, we compare three RoBERTa-architecture encoders that share architecture and parameter count (Table 1). Each backbone is fine-tuned end-to-end with an identical classification head; no encoder layers are frozen.

Short HF identifier	Pre-training corpus	Role
roberta-base [6]	English text only	Control
microsoft/codebert-base [3]	Code + NL bimodal	Code-aware
microsoft/graphcodebert-base [4]	Code + NL + data-flow	Structure-aware

Table 1: Backbones compared in the experiment

2.4 Training Protocol

We adopt 5-fold stratified cross-validation repeated across 5 random seeds, yielding 25 fold-runs per backbone and 75 in total. Each fold:

- splits a 90/10 stratified internal validation set used solely for early stopping;
- oversamples the minority class to match the majority count;
- trains with class-weighted cross-entropy (inverse frequency) plus label smoothing of 0.05;
- early-stops on macro-F1 with patience 4 via a custom `ManualEarlyStoppingCallback`.

Hyperparameters: learning rate 2×10^{-5} , batch size 8, weight decay 0.05, warmup ratio 0.15, dropout 0.2, max 12 epochs are held constant across all three backbones.

2.5 Tech Stack

The implementation uses PyTorch [8] and the HuggingFace `transformers`, `datasets`, and `accelerate` libraries [9] for tokenization, model loading, and the Trainer loop; `scikit-learn` [10] for splits and metric primitives; SciPy for paired statistical tests; and `matplotlib/seaborn` for figures. Reusable code is organised into six flat modules (`data_utils.py`, `model_utils.py`, `train_utils.py`, `evaluate.py`, `plotting.py`, plus `__init__.py`) so each module addresses a single concern.

2.6 Major Implementation Challenges

Two non-trivial issues surfaced during development. First, the HuggingFace `EarlyStoppingCallback` requires `load_best_model_at_end=True` and therefore checkpoint saving; with 75 fold-runs this exhausted the available Kaggle disk quota. We addressed this by writing a `ManualEarlyStoppingCallback` that tracks macro-F1 in memory and signals the trainer to stop without ever writing a checkpoint, and by setting `save_strategy="no"` plus deleting the trainer’s per-fold scratch directory at the end of each fold. Second, the modest dataset size (233 records) makes a single train/test split high-variance; the 5×5 CV protocol is what produces the tight per-fold spread reported below.

2.7 Deviations from Midterm Proposal

Three changes were made between the approved midterm proposal and the final implementation, each documented for honesty:

1. **Backbone family.** The proposal specified Llama 3.1 8B and DeepSeek-Coder 6.7B with parameter-efficient fine-tuning. At the available data scale, decoder-style models exhibited optimisation instability that obscured the design variables of interest. The implementation uses 125M-parameter RoBERTa-architecture encoders, which converge reliably and permit a controlled comparison across pre-training corpora.
2. **Task formulation.** The proposal targeted a five-class taxonomy. Initial multi-class experiments showed labels concentrated in a small number of classes, making fine-grained classification under-determined at this scale. The task was re-scoped to binary classical-vs-quantum classification.
3. **Evaluation methodology.** The proposal specified a single 70/10/20 split. The implementation uses 5-fold cross-validation repeated across five seeds, producing more stable point estimates and supporting paired Welch’s *t*-tests on matched folds.

3. Results and Evaluation

3.1 Quantitative Metrics

Table 2 reports the mean $\pm$ standard deviation of accuracy, macro-F1, and ROC-AUC across the 25 matched fold-runs per backbone, alongside pooled metrics computed from concatenated predictions across all folds. CodeBERT attained the highest mean accuracy and macro-F1; GraphCodeBERT attained the highest mean and pooled ROC-AUC; all three exceeded the random-classifier baseline by a wide margin (Figure 3).

Model	Accuracy	Macro-F1	ROC-AUC	Pool. Acc	Pool. F1	Pool. AUC
RoBERTa	0.764 ± 0.061	0.754 ± 0.066	0.858 ± 0.048	0.764	0.756	0.837
CodeBERT	0.767 ± 0.057	0.763 ± 0.056	0.855 ± 0.044	0.767	0.764	0.840
GraphCodeBERT	0.758 ± 0.058	0.756 ± 0.058	0.860 ± 0.047	0.758	0.757	0.854

Table 2: Cross-validation and pooled performance summary

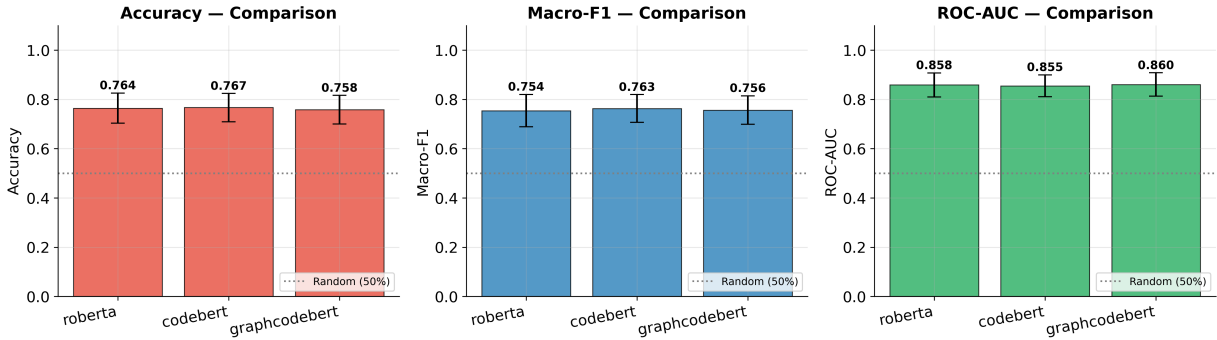
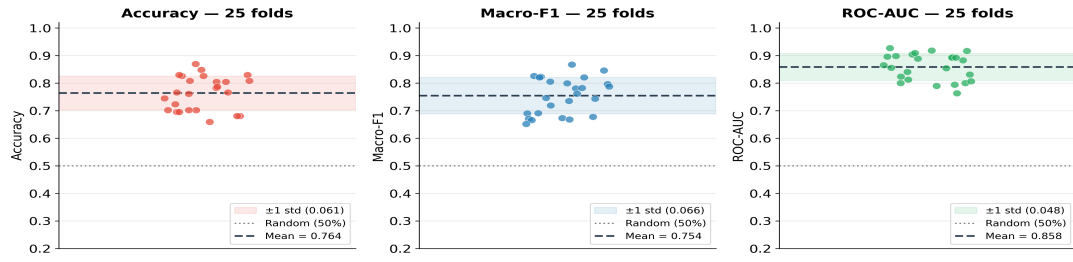


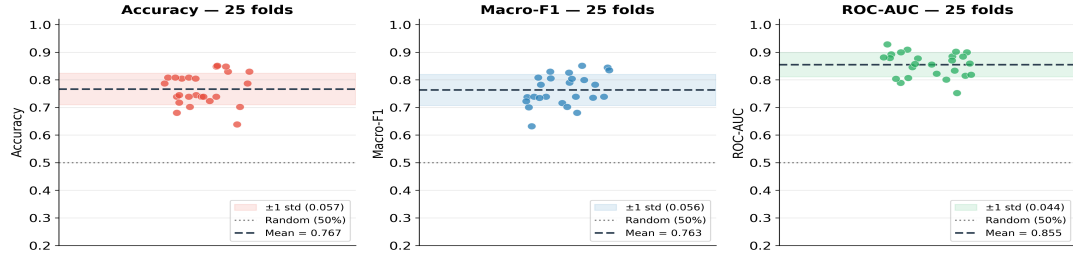
Figure 3: Cross-backbone performance comparison

Figure 4 shows the per-fold scatter of accuracy, macro-F1, and ROC-AUC for each backbone across all 25 folds, with the mean and ± 1 SD band overlaid. The within-backbone spread is comparable across all three models, and every fold sits well above the random (50%) baseline.

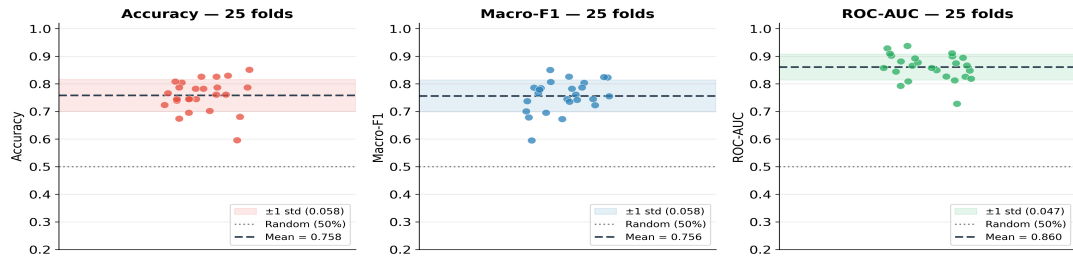
All three backbones met the proposal’s accuracy target of $\geq 70\%$. The pooled ROC curves Fig 5 in are tightly clustered with AUC values within 0.017 of one another (0.837–0.854). Per-backbone ROC curves with ± 1 SD bands remain narrow across the operating range, confirming stable ranking performance. The paired per-fold analysis (Figure 6) shows nearly horizontal connecting lines across backbones, confirming that fold-level difficulty transfers uniformly and no backbone systematically dominates.



(a) roberta-base



(b) codebert-base



(c) graphcodebert-base

Figure 4: Per-fold distributions of accuracy, macro-F1, and ROC-AUC across 25 fold-runs per backbone

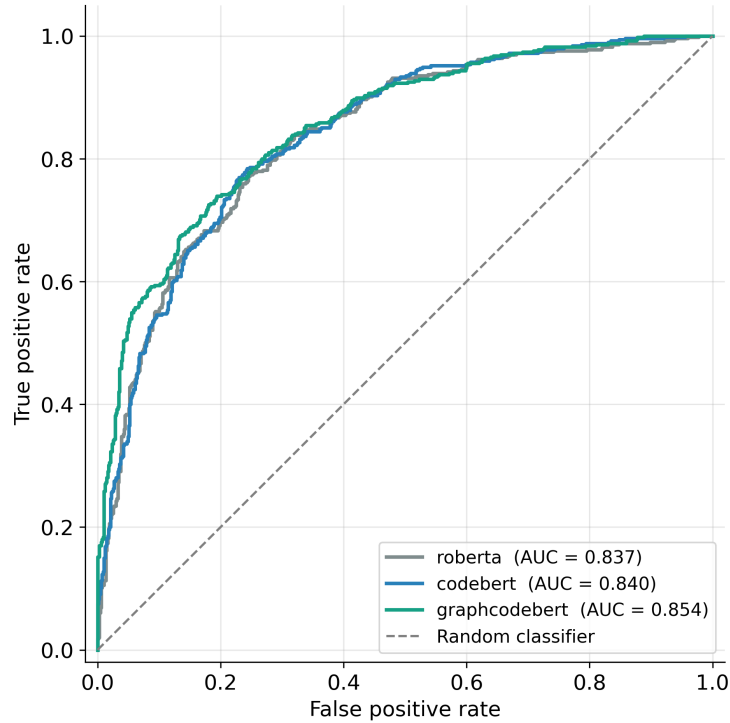


Figure 5: Pooled ROC comparison across backbones

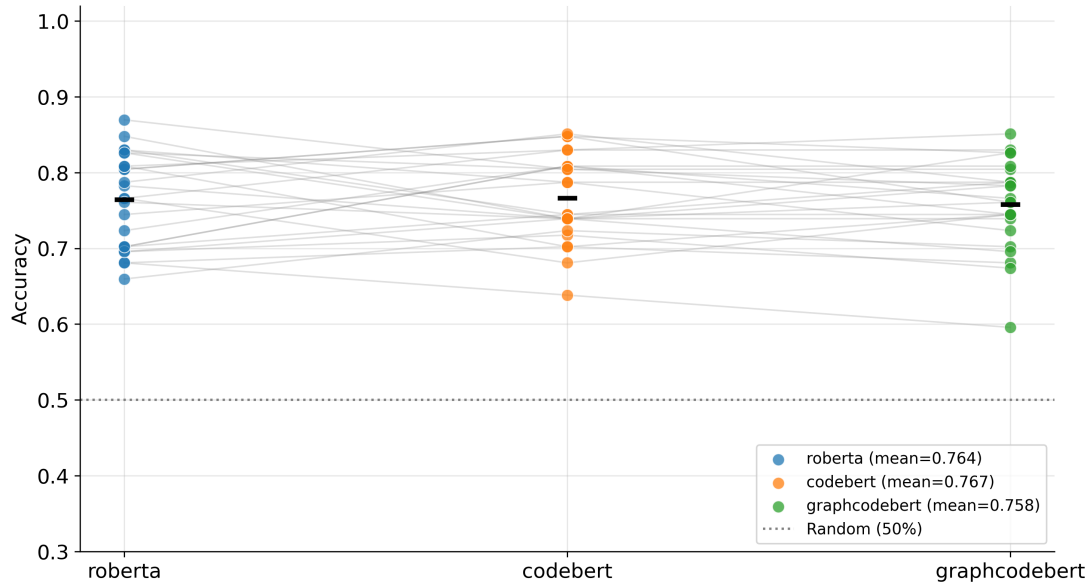


Figure 6: Matched-fold accuracy comparison across backbones

3.2 Confusion Matrices and Class Balance

Pooled confusion matrices (Figure 7) confirmed that the class-imbalance mitigation worked: recall on the minority quantum class was comparable to recall on the majority classical class for every backbone. CodeBERT achieved the highest balanced recall (75.2% classical, 78.6% quantum); RoBERTa showed a slight majority-class bias (82.4% classical vs. 68.3% quantum).

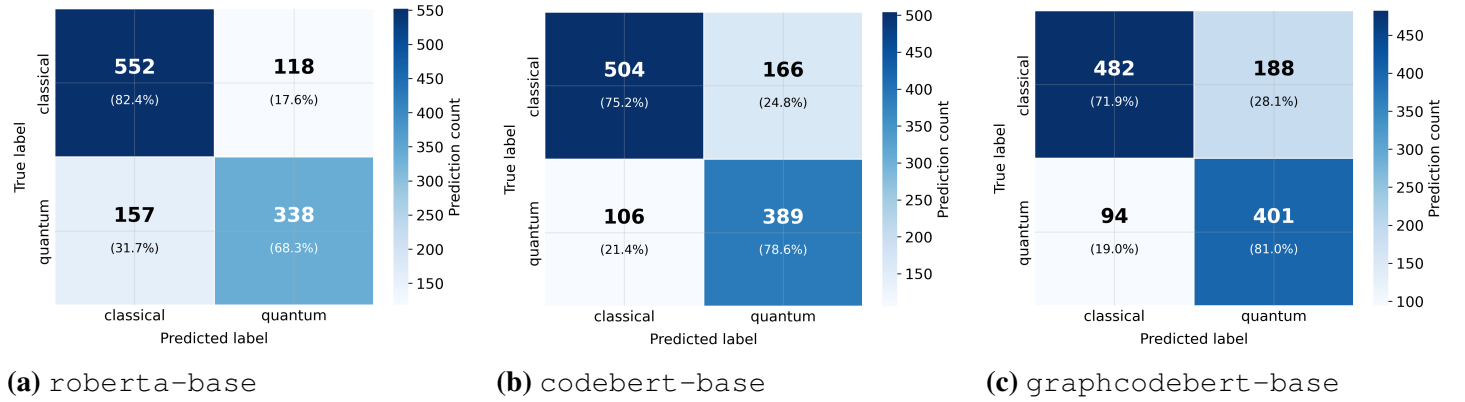


Figure 7: Pooled confusion matrices across 25 folds per backbone

3.3 Training Dynamics

Figure 8 shows mean validation loss and accuracy curves across epochs, averaged over 25 folds per backbone. All three converge by epoch 6–7. GraphCodeBERT reaches the lowest validation loss earliest, but the late-training plateau is shared across backbones and the differences at the final epoch are within fold-level noise.

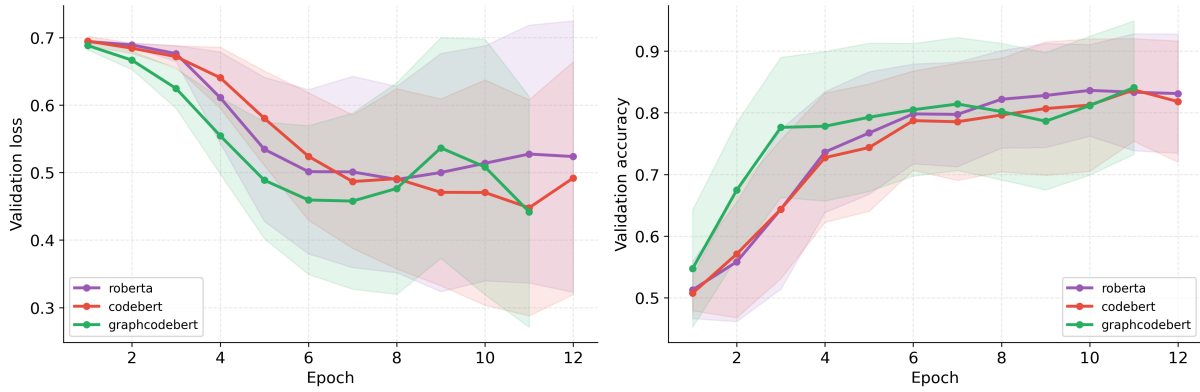


Figure 8: Cross-backbone training dynamics

3.4 Qualitative Observations

Two consistent misclassification patterns appeared across all three backbones. First, classical infrastructure issues involving Qiskit-specific module imports were occasionally predicted as quantum, suggesting the models relied on surface-level vocabulary cues. Second, quantum bugs with very short descriptions and no code snippet were more frequently predicted as classical, indicating that input length and the presence of code are influential discriminative signals. These patterns suggest that richer, code-heavy inputs would likely improve performance and better leverage code-aware pre-training.

4. Discussion

4.1 What Worked Well

The combination of class-weighted cross-entropy loss and per-fold minority oversampling successfully prevented majority-class collapse, producing balanced recall across both classes for every backbone. Without this dual strategy, early experiments collapsed onto predicting classical for all inputs. The custom ManualEarlyStopping-Callback solved the disk-quota problem cleanly and allowed the full 75-fold experiment to complete without interruption.

4.2 What Did Not Work as Expected

The central hypothesis of the study that code-aware pre-training would confer a measurable advantage for quantum bug classification was not supported. All three backbones produced statistically indistinguishable results on every metric (Table 2, Figures 3 and 6). One plausible explanation is that the corpus is dominated by natural-language descriptions rather than actual code; at this input distribution, code-aware pre-training offers no advantage because there is little code to leverage. A second explanation is that 233 samples is simply too small to expose the performance gap that would likely emerge at larger scale.

4.3 Limitations

- **Dataset scope.** The corpus is Qiskit-centric; generalisation to Cirq, Q#, or PennyLane is untested.
- **Input modality.** The corpus is dominated by prose descriptions rather than executable code, which may suppress the benefit of code-aware pre-training.
- **Fixed hyperparameters.** Backbone-specific tuning was not performed, so relative orderings could change under an optimised sweep.
- **Binary scope.** Re-scoping from five-class to binary classification simplifies the task; the practical utility for fine-grained triage remains untested.

4.4 Future Work

Three directions follow naturally from this work. First, dataset expansion through automated code perturbation and curated GitHub issue mining could provide additional labelled samples and may reveal backbone differences that are obscured at the current scale. Second, implementing prompt-only and retrieval-augmented generation (RAG) baselines would complete a four-method comparison and situate fine-tuning relative to inference-time alternatives. Third, applying parameter-efficient fine-tuning (LoRA / QLoRA [5, 2]) to a small generative model would enable structured output taxonomy class, suspected code location, and natural-language justification making the system actionable for developers rather than merely a binary classifier.

5. Conclusion

Key findings. This project fine-tuned three 125M-parameter encoder backbones on a Qiskit-centric quantum bug corpus and evaluated them under a rigorous 5-fold $\times$ 5-seed cross-validation protocol. All three models met the target accuracy of $\geq 70\%$ and achieved ROC-AUC near 0.86, with no statistically significant differences between any pair. The headline finding is methodological: at this dataset scale, the choice of pre-training corpus matters less than dataset construction and class-imbalance handling.

Personal learnings. Three practical lessons emerged. First, controlled experimental design pays off even when the result is negative: the null finding is defensible only because architecture, capacity, hyperparameters, and preprocessing were all held fixed. Second, class-imbalance handling is not optional at small data scales dropping either the weighted loss or the oversampling caused the model to collapse to majority-class prediction. Third, infrastructure constraints are real engineering decisions: rebuilding early stopping to avoid checkpoint writes was the difference between an experiment that completed and one that would have crashed halfway through.

Connection to applied LLM development. For domain-specific text classification with only a few hundred training samples, the choice of pre-trained encoder matters less than how the data is curated and how class imbalance is handled. Future gains are most likely to come from expanding the labelled dataset and from compar-

ing fine-tuning against prompt-based and retrieval-augmented approaches, which remain open directions for this system.

References

- [1] Campos, J., & Souto, A. (2021). QBugs: A collection of reproducible bugs in quantum algorithms. *arXiv preprint arXiv:2103.16968*. <https://doi.org/10.48550/arXiv.2103.16968>
- [2] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized language models. *NeurIPS*, 36, 10088–10115. <https://arxiv.org/abs/2305.14314>
- [3] Feng, Z., Guo, D., Tang, D., et al. (2020). CodeBERT: A pre-trained model for programming and natural languages. *Findings of EMNLP 2020*, 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- [4] Guo, D., Ren, S., Lu, S., et al. (2021). GraphCodeBERT: Pre-training code representations with data flow. *ICLR 2021*. <https://arxiv.org/abs/2009.08366>
- [5] Hu, E. J., Shen, Y., Wallis, P., et al. (2022). LoRA: Low-rank adaptation of large language models. *ICLR 2022*. <https://arxiv.org/abs/2106.09685>
- [6] Liu, Y., Ott, M., Goyal, N., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*. <https://arxiv.org/abs/1907.11692>
- [7] Zhao, P., Zhao, J., Miao, Z., & Lan, S. (2021). Bugs4Q: A benchmark of real bugs for quantum programs. *ASE 2021*, 1373–1376. <https://doi.org/10.1109/ASE51524.2021.9678908>
- [8] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS 32*. <https://arxiv.org/abs/1912.01703>
- [9] Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-art natural language processing. *EMNLP 2020: System Demonstrations*, 38–45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- [10] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>